



BRAINWARE UNIVERSITY

School of Engineering

Department of Computer Science and Engineering – Cyber Security & Data Science

398, Ramkrishnapur Road, Barasat, North 24 Parganas, Kolkata – 700 125

LAB MANUAL

Academic Year: 2025-26 EVEN SEMESTER

Programme (UG/PG)	: UG
Semester	: 4th
Course Code	: BTY49111
Course Title	: Object Oriented Programming Lab
Credit	: 1.5
Contacts	: 3P



BRAINWARE UNIVERSITY

School of Engineering

Department of Computer Science and Engineering – Cyber Security & Data Science

398, Ramkrishnapur Road, Barasat, North 24 Parganas, Kolkata – 700 125

SYLLABUS

Course Name: Object Oriented Programming Lab

Course Code: PCC-CSD493

Contact: 3P

Credits: 1.5

No. of Periods: 45

Module I: Basics of Java

[3H]

Simple Java program with a class and object.

Module II: Concept of string

[9H]

Method Overloading, user input using different classes, String methods, StringBuffer methods, command line argument.

Module III: Concept of Inheritance and package

[15H]

Constructor, Inheritance, abstract class, interface, package.

Module IV: Exception handling

[9H]

Exception using try-catch block, user defined exception and thread life cycle.

Module V: Implementation of AWT

[9H]

AWT and Swing programming.



BRAINWARE UNIVERSITY

School of Engineering

Department of Computer Science and Engineering – Cyber Security & Data Science

398, Ramkrishnapur Road, Barasat, North 24 Parganas, Kolkata – 700 125

LIST OF EXPERIMENTS

1. Write a Java program to display student details using class and object.

```
class Student
{
    void display()
    {
        System.out.println("Name: Rahul, Roll: 101");
    }
    public static void main(String[] args)
    {
        Student s = new Student();
        s.display();
    }
}
```

2. Write a Java program to display employee details using class and object.

```
class Employee
{
    int empid = 12;
    String dept = "IT";
    void show()
    {
        System.out.println("Employee ID: " + empid + ", Dept:" + dept);
    }
}
class Main()
{
    public static void main(String[] args)
    {
        Employee e = new Employee();
        e.show();
    }
}
```

3. Write a Java program demonstrating method overloading using addition operation.

```
class AddNumbers
{
    int add(int a, int b)
    {
        return a + b;
    }
}
```



BRAINWARE UNIVERSITY

School of Engineering

Department of Computer Science and Engineering – Cyber Security & Data Science

398, Ramkrishnapur Road, Barasat, North 24 Parganas, Kolkata – 700 125

```
}
int add(int a, int b, int c)
{
    return a + b + c;
}
public static void main(String[] args)
{
    AddNumbers obj = new AddNumbers();
    System.out.println(obj.add(10, 20));
    System.out.println(obj.add(10, 20, 30));
}
}
```

4. Write a Java program to calculate area of different shapes using method overloading.

```
class Area
{
    double area(double r)
    {
        return 3.14 * r * r;
    }
    int area(int l, int b)
    {
        return l * b;
    }
    public static void main(String[] args)
    {
        Area a = new Area();
        System.out.println(a.area(5));
        System.out.println(a.area(4, 6));
    }
}
```

5. Write a Java program to accept user input using the Scanner class.

```
import java.util.Scanner;

class ScannerInput
{
    void read()
    {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter number: ");
        int n = sc.nextInt();
    }
}
```



BRAINWARE UNIVERSITY

School of Engineering

Department of Computer Science and Engineering – Cyber Security & Data Science

398, Ramkrishnapur Road, Barasat, North 24 Parganas, Kolkata – 700 125

```
        System.out.println("Number: " + n);
    }
```

```
public static void main(String[] args)
{
    new ScannerInput().read();
}
}
```

6. Write a Java program to accept an employee details (empid and name) using the BufferedReader class.

```
import java.io.*;
```

```
class BufferedInput
{
    int empid;
    try
    {
        BufferedReader br = new BufferedReader(new InputStreamReader(System.in));
        System.out.print("Enter empid: ");
        empid = Integer.parseInt(br.readLine());
        System.out.print("Enter name: ");
        String name = br.readLine();

        System.out.println("EmpID:" + empid + "Name:" + name);
    }
    catch(IOException e)
    {
    }
    public static void main(String[] args)
    {
        new BufferedInput().read();
    }
}
```

7. Write a Java program demonstrating all commonly used String class methods.

```
class StringMethodsDemo
{
    void demonstrateMethods()
    {
        String s1 = "Java Programming";
        String s2 = "java programming";
    }
}
```



BRAINWARE UNIVERSITY

School of Engineering

Department of Computer Science and Engineering – Cyber Security & Data Science

398, Ramkrishnapur Road, Barasat, North 24 Parganas, Kolkata – 700 125

```
String s3 = " Welcome to Java ";

// Length
System.out.println("Length: " + s1.length());

// Character at index
System.out.println("Char at index 5: " + s1.charAt(5));

// Compare strings
System.out.println("Equals: " + s1.equals(s2));
System.out.println("Equals Ignore Case: " + s1.equalsIgnoreCase(s2));

// Compare lexicographically
System.out.println("Compare To: " + s1.compareTo(s2));

// Case conversion
System.out.println("Upper Case: " + s1.toUpperCase());
System.out.println("Lower Case: " + s1.toLowerCase());

// Substring
System.out.println("Substring (5): " + s1.substring(5));
System.out.println("Substring (5, 11): " + s1.substring(5, 11));

// Replace
System.out.println("Replace: " + s1.replace("Java", "Python"));

// Contains
System.out.println("Contains 'Program': " + s1.contains("Program"));

// Starts and Ends with
System.out.println("Starts with 'Java': " + s1.startsWith("Java"));
System.out.println("Ends with 'ing': " + s1.endsWith("ing"));

// Index methods
System.out.println("Index of 'a': " + s1.indexOf('a'));
System.out.println("Last Index of 'a': " + s1.lastIndexOf('a'));

// Trim
System.out.println("Trimmed String: " + s3.trim() + "");
```



BRAINWARE UNIVERSITY

School of Engineering

Department of Computer Science and Engineering – Cyber Security & Data Science

398, Ramkrishnapur Road, Barasat, North 24 Parganas, Kolkata – 700 125

```
// Split
String[] words = s1.split(" ");
System.out.println("Split words:");
for (String w : words)
{
    System.out.println(w);
}

// ValueOf
int num = 100;
String numStr = String.valueOf(num);
System.out.println("ValueOf: " + numStr);
}

public static void main(String[] args)
{
    StringMethodsDemo obj = new StringMethodsDemo();
    obj.demonstrateMethods();
}
}
```

8. Write a Java program that accepts a string through command line arguments and demonstrates the use of various StringBuffer methods such as append(), reverse(), insert(), and replace().

```
class StringBufferCommandLine
{

    void processString(String input)
    {
        StringBuffer sb = new StringBuffer(input);

        // Append text
        sb.append(" Technology");
        System.out.println("After append: " + sb);

        // Insert text
        sb.insert(0, "Java ");
        System.out.println("After insert: " + sb);

        // Replace part of string
        sb.replace(5, 9, "Language");
    }
}
```



BRAINWARE UNIVERSITY

School of Engineering

Department of Computer Science and Engineering – Cyber Security & Data Science

398, Ramkrishnapur Road, Barasat, North 24 Parganas, Kolkata – 700 125

```
        System.out.println("After replace: " + sb);

        // Reverse the string
        sb.reverse();
        System.out.println("After reverse: " + sb);
    }

    public static void main(String[] args)
    {
        if (args.length == 0)
        {
            System.out.println("Please provide a string as command line argument.");
            return;
        }

        StringBufferCommandLine obj = new StringBufferCommandLine();
        obj.processString(args[0]);
    }
}
```

9. Use Case Scenario

In a college management system, when a new department record is created without providing any initial details, the system should automatically assign default values such as department name and total number of students. This ensures that the object is initialized with meaningful data even when no information is passed by the user.

Problem Statement

Write a Java program to demonstrate the use of a default constructor that initializes an object with default values and displays the details.

```
class Department
{
    String deptName;
    int totalStudents;

    // Default constructor
    Department()
    {
        deptName = "Computer Science";
        totalStudents = 60;
    }
}
```




BRAINWARE UNIVERSITY

School of Engineering

Department of Computer Science and Engineering – Cyber Security & Data Science

398, Ramkrishnapur Road, Barasat, North 24 Parganas, Kolkata – 700 125

```
// Method to display details
void displayDetails()
{
    System.out.println("Department Name: " + deptName);
    System.out.println("Total Students: " + totalStudents);
}

public static void main(String[] args)
{
    // Object creation invokes default constructor
    Department dept = new Department();
    dept.displayDetails();
}
}
```

10. Use Case Scenario

In a student registration system, student details such as name and roll number must be provided at the time of object creation. To ensure that each student object is initialized with user-specific data, a parameterized constructor is used.

Problem Statement

Write a Java program to demonstrate the use of a parameterized constructor that initializes object data at the time of object creation and displays the details.

```
class Student
{
    String name;
    int rollNo;

    // Parameterized constructor
    Student(String n, int r)
    {
        name = n;
        rollNo = r;
    }

    // Method to display student details
    void displayDetails()
    {
        System.out.println("Student Name: " + name);
    }
}
```



BRAINWARE UNIVERSITY

School of Engineering

Department of Computer Science and Engineering – Cyber Security & Data Science

398, Ramkrishnapur Road, Barasat, North 24 Parganas, Kolkata – 700 125

```
        System.out.println("Roll Number: " + rollNo);
    }

    public static void main(String[] args)
    {
        // Creating object using parameterized constructor
        Student s1 = new Student("Amit", 101);
        s1.displayDetails();
    }
}
```

11. Use Case Scenario

In an employee management system, basic employee information such as employee ID and name is stored in a base class. Additional details like department and designation are handled by a derived class. To reuse the common properties and methods of the base class, single-level inheritance is applied.

Problem Statement

Write a Java program to illustrate single-level inheritance where a child class inherits properties and methods from a parent class and displays employee details.

```
class Employee
{
    int empId;
    String name;

    // Method to initialize employee details
    void setEmployee(int id, String n)
    {
        empId = id;
        name = n;
    }
}

// Child class inheriting Employee
class Department extends Employee
{
    String deptName;

    // Method to initialize department details
    void setDepartment(String d)
```



BRAINWARE UNIVERSITY

School of Engineering

Department of Computer Science and Engineering – Cyber Security & Data Science

398, Ramkrishnapur Road, Barasat, North 24 Parganas, Kolkata – 700 125

```
{
    deptName = d;
}

// Method to display complete details
void displayDetails()
{
    System.out.println("Employee ID: " + empId);
    System.out.println("Employee Name: " + name);
    System.out.println("Department: " + deptName);
}

public static void main(String[] args)
{
    Department emp = new Department();
    emp.setEmployee(101, "Rahul");
    emp.setDepartment("IT");
    emp.displayDetails();
}
}
```

12. Use Case Scenario

In an academic management system, a Person has basic details such as name and age. A Student is a type of person who has additional information like roll number. A Result class further extends Student to store and display academic performance details such as marks and grade. This hierarchy demonstrates multi-level inheritance, where each level adds new functionality.

Problem Statement

Write a Java program to illustrate multi-level inheritance where a derived class inherits properties and methods across multiple levels and displays complete student academic details.

```
class Person {

    String name;
    int age;

    // Method to set person details
    void setPerson(String n, int a) {
        name = n;
        age = a;
    }
}
```



BRAINWARE UNIVERSITY

School of Engineering

Department of Computer Science and Engineering – Cyber Security & Data Science

398, Ramkrishnapur Road, Barasat, North 24 Parganas, Kolkata – 700 125

```
}

// First level inheritance
class Student extends Person {

    int rollNo;

    // Method to set student details
    void setStudent(int r) {
        rollNo = r;
    }
}

// Second level inheritance
class Result extends Student {

    int marks;

    // Method to set result details
    void setResult(int m) {
        marks = m;
    }

    // Method to display all details
    void displayDetails() {
        System.out.println("Name: " + name);
        System.out.println("Age: " + age);
        System.out.println("Roll Number: " + rollNo);
        System.out.println("Marks: " + marks);
    }

    public static void main(String[] args) {
        Result r = new Result();
        r.setPerson("Anita", 20);
        r.setStudent(105);
        r.setResult(88);
        r.displayDetails();
    }
}
```



BRAINWARE UNIVERSITY

School of Engineering

Department of Computer Science and Engineering – Cyber Security & Data Science

398, Ramkrishnapur Road, Barasat, North 24 Parganas, Kolkata – 700 125

13. Use Case Scenario

In a graphics application, different shapes need to be drawn on the screen. While all shapes share a common behavior of being drawable, the actual drawing logic differs for each shape. To enforce this structure, an abstract class is used with an abstract method that must be implemented by all derived shape classes.

Problem Statement

Write a Java program to demonstrate the use of an abstract class containing an abstract method, where the method is implemented by a derived class.

```
abstract class Shape {  
  
    // Abstract method  
    abstract void draw();  
}  
  
// Derived class implementing abstract method  
class Circle extends Shape {  
  
    void draw() {  
        System.out.println("Drawing a Circle");  
    }  
  
    public static void main(String[] args) {  
        Shape s = new Circle();  
        s.draw();  
    }  
}
```

14. Use Case Scenario

In a vehicle management system, all vehicles share some common behavior such as starting the engine, while other behaviors like calculating fuel efficiency vary depending on the type of vehicle. To model this situation, an abstract class is used that contains both concrete methods (common functionality) and abstract methods (vehicle-specific functionality).

Problem Statement

Write a Java program to demonstrate an abstract class containing both abstract and concrete methods, where the abstract method is implemented by a derived class.



BRAINWARE UNIVERSITY

School of Engineering

Department of Computer Science and Engineering – Cyber Security & Data Science

398, Ramkrishnapur Road, Barasat, North 24 Parganas, Kolkata – 700 125

```
abstract class Vehicle {  
  
    // Concrete method  
    void start() {  
        System.out.println("Vehicle engine started");  
    }  
  
    // Abstract method  
    abstract void mileage();  
}  
  
// Derived class implementing abstract method  
class Car extends Vehicle {  
  
    void mileage() {  
        System.out.println("Car mileage is 15 km/l");  
    }  
  
    public static void main(String[] args) {  
        Vehicle v = new Car();  
        v.start();  
        v.mileage();  
    }  
}
```

15. Use Case Scenario

In a printing system, different devices such as printers and scanners need to perform a common operation called print. To ensure that every device follows the same contract for printing functionality, a Java interface is used. Any class implementing this interface must provide its own implementation of the print operation.

Problem Statement

Write a Java program to demonstrate the implementation of a simple interface and its method in a concrete class.

```
// Interface declaration  
interface Printable {  
    void print();  
}
```



BRAINWARE UNIVERSITY

School of Engineering

Department of Computer Science and Engineering – Cyber Security & Data Science

398, Ramkrishnapur Road, Barasat, North 24 Parganas, Kolkata – 700 125

```
// Class implementing the interface
class Printer implements Printable {

    public void print() {
        System.out.println("Printing document...");
    }

    public static void main(String[] args) {
        Printable p = new Printer();
        p.print();
    }
}
```

16. Use Case Scenario

In an academic system, a teaching assistant performs both teaching activities and research activities. Teaching responsibilities and research responsibilities are defined separately. To allow a single class to inherit behaviors from more than one source without ambiguity, multiple inheritance is achieved using interfaces in Java.

Problem Statement

Write a Java program to demonstrate multiple inheritance using interfaces, where a class implements more than one interface.

```
// First interface
interface Teaching {
    void teach();
}

// Second interface
interface Research {
    void research();
}

// Class implementing multiple interfaces
class TeachingAssistant implements Teaching, Research {

    public void teach() {
        System.out.println("Teaching undergraduate students");
    }
}
```



BRAINWARE UNIVERSITY

School of Engineering

Department of Computer Science and Engineering – Cyber Security & Data Science

398, Ramkrishnapur Road, Barasat, North 24 Parganas, Kolkata – 700 125

```
public void research() {  
    System.out.println("Conducting research work");  
}
```

```
public static void main(String[] args) {  
    TeachingAssistant ta = new TeachingAssistant();  
    ta.teach();  
    ta.research();  
}
```

17. Use Case Scenario

In a large software project, related utility classes are grouped together to improve code organization, reusability, and maintainability. For example, all common mathematical utility functions can be placed inside a user-defined package so that they can be reused across multiple programs.

Problem Statement

Write a Java program to create a user-defined package and use a class from that package in another program.

Step 1: Create a User-Defined Package

File: mypack/Calculator.java

```
package mypack;
```

```
public class Calculator {  
  
    public void add(int a, int b) {  
        System.out.println("Sum = " + (a + b));  
    }  
}
```

Step 2: Use the User-Defined Package

File: TestPackage.java

```
import mypack.Calculator;
```

```
class TestPackage {  
  
    public static void main(String[] args) {  
        Calculator calc = new Calculator();  
        calc.add(10, 20);  
    }  
}
```




BRAINWARE UNIVERSITY

School of Engineering

Department of Computer Science and Engineering – Cyber Security & Data Science

398, Ramkrishnapur Road, Barasat, North 24 Parganas, Kolkata – 700 125

```
}  
}
```

Compilation & Execution

```
javac mypack/Calculator.java
```

```
javac TestPackage.java
```

```
java TestPackage
```

18. Use Case Scenario

In a banking application, common utility classes like Calculator, AccountUtils, or Validator are organized into a user-defined package. To perform operations in different modules of the application, these classes are imported and used wherever needed, demonstrating modular programming and code reuse.

Problem Statement

Write a Java program to demonstrate the use of a user-defined package by importing a class from the package and invoking its methods.

Step 1: Create a User-Defined Package (if not already created)

File: mypack/Calculator.java

```
package mypack;  
  
public class Calculator {  
  
    public int add(int a, int b) {  
        return a + b;  
    }  
  
    public int multiply(int a, int b) {  
        return a * b;  
    }  
}
```

Step 2: Use the Package in Another Program

File: UsePackageDemo.java

```
import mypack.Calculator;
```



BRAINWARE UNIVERSITY

School of Engineering

Department of Computer Science and Engineering – Cyber Security & Data Science

398, Ramkrishnapur Road, Barasat, North 24 Parganas, Kolkata – 700 125

```
class UsePackageDemo {  
  
    public static void main(String[] args) {  
        Calculator calc = new Calculator();  
  
        int sum = calc.add(15, 25);  
        int product = calc.multiply(5, 6);  
  
        System.out.println("Sum = " + sum);  
        System.out.println("Product = " + product);  
    }  
}
```

Compilation & Execution

```
javac mypack/Calculator.java
```

```
javac UsePackageDemo.java
```

```
java UsePackageDemo
```

19. Use Case Scenario

In a calculator application, users can perform division of two numbers. If the user enters zero as the divisor, the program may crash due to an arithmetic exception (division by zero). To handle this gracefully, a try-catch block is used to catch the exception and display a meaningful message instead of terminating abruptly.

Problem Statement

Write a Java program to demonstrate handling of arithmetic exceptions using a try-catch block.

```
import java.util.Scanner;  
  
class ArithmeticExceptionDemo {  
  
    void divideNumbers() {  
        Scanner sc = new Scanner(System.in);  
  
        System.out.print("Enter numerator: ");  
        int num = sc.nextInt();  
  
        System.out.print("Enter denominator: ");  
        int denom = sc.nextInt();
```



BRAINWARE UNIVERSITY

School of Engineering

Department of Computer Science and Engineering – Cyber Security & Data Science

398, Ramkrishnapur Road, Barasat, North 24 Parganas, Kolkata – 700 125

```
try {
int result = num / denom; // May throw ArithmeticException
System.out.println("Result = " + result);
} catch (ArithmeticException e) {
System.out.println("Error: Division by zero is not allowed.");
} finally {
System.out.println("Execution completed.");
}

sc.close();
}

public static void main(String[] args) {
ArithmeticExceptionDemo obj = new ArithmeticExceptionDemo();
obj.divideNumbers();
}
}
```

20. Use Case Scenario

In a student management system, marks of students are stored in an array. If a user tries to access a position outside the array bounds, it may cause the program to crash. To prevent this, an `ArrayIndexOutOfBoundsException` is caught using a try-catch block, allowing the program to continue executing safely.

Problem Statement

Write a Java program to demonstrate handling of array index out of bounds exception using a try-catch block.

```
class ArrayExceptionDemo {

void accessArray() {
int[] marks = {85, 90, 78, 92, 88};

try {
// Attempting to access an invalid index
System.out.println("Accessing element at index 7: " + marks[7]);
} catch (ArrayIndexOutOfBoundsException e) {
System.out.println("Error: Invalid array index accessed.");
}
```



BRAINWARE UNIVERSITY

School of Engineering

Department of Computer Science and Engineering – Cyber Security & Data Science

398, Ramkrishnapur Road, Barasat, North 24 Parganas, Kolkata – 700 125

```
    } finally {  
        System.out.println("Array access attempt completed.");  
    }  
}  
  
public static void main(String[] args) {  
    ArrayExceptionDemo obj = new ArrayExceptionDemo();  
    obj.accessArray();  
}  
}
```

21. Use Case Scenario

In a college admission system, students must be at least 18 years old to enroll. If a student is underage, the system should throw a custom exception to prevent registration. This demonstrates how user-defined exceptions can enforce business rules and validate data.

Problem Statement

Write a Java program to demonstrate the creation and use of a user-defined exception by validating student age.

```
// User-defined exception class  
class AgeException extends Exception {  
    AgeException(String message) {  
        super(message);  
    }  
}  
  
// Main class  
class UserDefinedExceptionDemo {  
  
    void checkAge(int age) throws AgeException {  
        if (age < 18) {  
            throw new AgeException("Student is underage. Admission not allowed.");  
        } else {  
            System.out.println("Student is eligible for admission.");  
        }  
    }  
}  
  
public static void main(String[] args) {
```



BRAINWARE UNIVERSITY

School of Engineering

Department of Computer Science and Engineering – Cyber Security & Data Science

398, Ramkrishnapur Road, Barasat, North 24 Parganas, Kolkata – 700 125

```
UserDefinedExceptionDemo obj = new UserDefinedExceptionDemo();
int studentAge = 16; // Example age

try {
    obj.checkAge(studentAge);
} catch (AgeException e) {
    System.out.println("Error: " + e.getMessage());
}
}
```

22. Use Case Scenario

In a banking application, a customer should not be able to withdraw an amount greater than the available balance. To enforce this rule, a user-defined exception is created to handle insufficient balance scenarios, preventing unauthorized withdrawals.

Problem Statement

Write a Java program to demonstrate the use of a user-defined exception for balance validation in a banking application.

```
// User-defined exception class
class InsufficientBalanceException extends Exception {
    InsufficientBalanceException(String message) {
        super(message);
    }
}

// Main class
class Bank {

    int balance = 5000;

    void withdraw(int amount) throws InsufficientBalanceException {
        if (amount > balance) {
            throw new InsufficientBalanceException("Insufficient balance. Withdrawal denied.");
        } else {
            balance -= amount;
        }
    }
}
```



BRAINWARE UNIVERSITY

School of Engineering

Department of Computer Science and Engineering – Cyber Security & Data Science

398, Ramkrishnapur Road, Barasat, North 24 Parganas, Kolkata – 700 125

```
        System.out.println("Withdrawal successful. Remaining balance: " + balance);
    }
}

public static void main(String[] args) {
    Bank account = new Bank();
    int withdrawAmount = 6000; // Example withdrawal

    try {
        account.withdraw(withdrawAmount);
    } catch (InsufficientBalanceException e) {
        System.out.println("Error: " + e.getMessage());
    }
}
}
```

23. Use Case Scenario

In a file downloading application, a task such as downloading a file runs independently of the main program. Understanding the life cycle of a thread helps ensure proper execution and termination of such tasks. A single thread goes through states like new, runnable, running, and terminated.

Problem Statement

Write a Java program to demonstrate the life cycle of a single thread using the Thread class and the run() method.

```
class SingleThreadDemo extends Thread {

    public void run() {
        System.out.println("Thread is running...");
        for (int i = 1; i <= 5; i++) {
            System.out.println("Count: " + i);
            try {
                Thread.sleep(500); // Pause for 0.5 second
            } catch (InterruptedException e) {
                System.out.println("Thread interrupted");
            }
        }
    }
}
```



BRAINWARE UNIVERSITY

School of Engineering

Department of Computer Science and Engineering – Cyber Security & Data Science

398, Ramkrishnapur Road, Barasat, North 24 Parganas, Kolkata – 700 125

```
    }
    System.out.println("Thread execution completed.");
}

public static void main(String[] args) {
    // Thread in new state
    SingleThreadDemo t1 = new SingleThreadDemo();
    System.out.println("Thread created (New State)");

    // Thread moves to runnable state
    t1.start(); // Thread starts and moves to running state

    // Main thread continues execution
    System.out.println("Main thread continues execution");
}
}
```

24. Use Case Scenario

In a banking system, multiple tasks such as updating account balances, sending notifications, and logging transactions run simultaneously. Understanding the life cycle of multiple threads allows these tasks to execute concurrently without blocking each other, ensuring efficient system performance.

Problem Statement

Write a Java program to demonstrate the life cycle of multiple threads using the Thread class. Each thread should perform a simple counting task.

```
class MultiThreadDemo extends Thread {

    String threadName;

    MultiThreadDemo(String name) {
        threadName = name;
    }

    public void run() {
        System.out.println(threadName + " started.");
        for (int i = 1; i <= 5; i++) {
```



BRAINWARE UNIVERSITY

School of Engineering

Department of Computer Science and Engineering – Cyber Security & Data Science

398, Ramkrishnapur Road, Barasat, North 24 Parganas, Kolkata – 700 125

```
        System.out.println(threadName + " count: " + i);
    try {
        Thread.sleep(500); // Pause for 0.5 second
    } catch (InterruptedException e) {
        System.out.println(threadName + " interrupted.");
    }
}
System.out.println(threadName + " completed.");
}

public static void main(String[] args) {
    // Create multiple threads
    MultiThreadDemo t1 = new MultiThreadDemo("Thread-1");
    MultiThreadDemo t2 = new MultiThreadDemo("Thread-2");

    // Start threads (move from new to runnable)
    t1.start();
    t2.start();

    System.out.println("Main thread continues execution...");
}
}
```

25. Use Case Scenario

In a desktop application, the user needs a basic window to display information or interact with the system. Using AWT (Abstract Window Toolkit), a simple GUI window (Frame) can be created as the main interface for the application.

Problem Statement

Write a Java program to create a basic GUI using AWT Frame with a title.

```
import java.awt.*; // Import AWT package

class BasicAWTFrame {

    BasicAWTFrame() {
        // Create a Frame
```




BRAINWARE UNIVERSITY

School of Engineering

Department of Computer Science and Engineering – Cyber Security & Data Science

398, Ramkrishnapur Road, Barasat, North 24 Parganas, Kolkata – 700 125

```
Frame f = new Frame("My First AWT Frame");

// Set frame size
f.setSize(400, 300);

// Set frame visibility
f.setVisible(true);
}

public static void main(String[] args) {
    // Create an instance of BasicAWTFrame
    new BasicAWTFrame();
}
}
```

26. Use Case Scenario

In a desktop application, users need an interactive button to perform actions such as submitting a form or triggering a task. Using AWT, a Button component can be added to a Frame to enable user interaction.

Problem Statement

Write a Java program to create a GUI using AWT Frame with a Button component.

```
import java.awt.*;

class AWTButtonDemo {

    AWTButtonDemo() {
        // Create a Frame
        Frame f = new Frame("AWT Button Example");

        // Create a Button
        Button b = new Button("Click Me");

        // Set Button position and size
        b.setBounds(100, 100, 80, 30); // x, y, width, height

        // Add Button to Frame
```



BRAINWARE UNIVERSITY

School of Engineering

Department of Computer Science and Engineering – Cyber Security & Data Science

398, Ramkrishnapur Road, Barasat, North 24 Parganas, Kolkata – 700 125

```
f.add(b);

// Set Frame size and layout
f.setSize(400, 300);
f.setLayout(null); // Using no layout managers
f.setVisible(true);
}

public static void main(String[] args) {
    new AWTButtonDemo();
}
}
```

27. Use Case Scenario

In a desktop application, a user clicks a button to perform an action, such as displaying a message. Event handling is required to respond to user actions. Using AWT, the program can detect a button click event and execute the corresponding action.

Problem Statement

Write a Java program to create a GUI using AWT with a Button component and demonstrate event handling for a button click.

```
import java.awt.*;
import java.awt.event.*;

class ButtonClickDemo implements ActionListener {

    Frame f;
    Button b;

    ButtonClickDemo() {
        // Create a Frame
        f = new Frame("Button Click Event Example");

        // Create a Button
        b = new Button("Click Me");
```



BRAINWARE UNIVERSITY

School of Engineering

Department of Computer Science and Engineering – Cyber Security & Data Science

398, Ramkrishnapur Road, Barasat, North 24 Parganas, Kolkata – 700 125

```
// Set Button position and size
b.setBounds(100, 100, 80, 30);

// Add action listener to the button
b.addActionListener(this);

// Add button to frame
f.add(b);

// Set Frame size and layout
f.setSize(400, 300);
f.setLayout(null);
f.setVisible(true);
}

// Override actionPerformed method
public void actionPerformed(ActionEvent e) {
    System.out.println("Button clicked!");
}

public static void main(String[] args) {
    new ButtonClickDemo();
}
}
```

28. Use Case Scenario

In a desktop application, the user may click the close button on a window to exit the application. To gracefully handle window closing events, the program should respond and terminate the application properly, preventing abrupt termination or resource leaks. Using AWT, this can be achieved with WindowListener.

Problem Statement

Write a Java program to create an AWT Frame and handle the window closing event to terminate the application gracefully.



BRAINWARE UNIVERSITY

School of Engineering

Department of Computer Science and Engineering – Cyber Security & Data Science

398, Ramkrishnapur Road, Barasat, North 24 Parganas, Kolkata – 700 125

```
import java.awt.*;
import java.awt.event.*;

class WindowClosingDemo {

    WindowClosingDemo() {
        // Create a Frame
        Frame f = new Frame("Window Closing Event Example");

        // Set Frame size
        f.setSize(400, 300);

        // Add WindowListener to handle close event
        f.addWindowListener(new WindowAdapter() {
            public void windowClosing(WindowEvent e) {
                System.out.println("Window is closing...");
                f.dispose(); // Close the window
            }
        });

        // Set layout and make visible
        f.setLayout(null);
        f.setVisible(true);
    }

    public static void main(String[] args) {
        new WindowClosingDemo();
    }
}
```

29. Use Case Scenario

In a modern desktop application, users need interactive components like buttons to trigger actions. Swing provides a richer set of GUI components than AWT, including JButton, which allows more customization and better look-and-feel.

Problem Statement

Write a Java program to create a GUI using Swing and add a JButton component.



BRAINWARE UNIVERSITY

School of Engineering

Department of Computer Science and Engineering – Cyber Security & Data Science

398, Ramkrishnapur Road, Barasat, North 24 Parganas, Kolkata – 700 125

```
import javax.swing.*;

class SwingButtonDemo {

    SwingButtonDemo() {
        // Create a JFrame
        JFrame frame = new JFrame("Swing Button Example");

        // Create a JButton
        JButton button = new JButton("Click Me");

        // Set button position and size
        button.setBounds(100, 100, 100, 40);

        // Add button to frame
        frame.add(button);

        // Set frame size, layout, and visibility
        frame.setSize(400, 300);
        frame.setLayout(null);
        frame.setVisible(true);

        // Close operation
        frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
    }

    public static void main(String[] args) {
        new SwingButtonDemo();
    }
}
```

30. Use Case Scenario

In a student registration system, users need to enter information through a graphical form. Using Swing, we can create a simple form with components like JLabel, JTextField, and JButton to collect input data.



BRAINWARE UNIVERSITY

School of Engineering

Department of Computer Science and Engineering – Cyber Security & Data Science

398, Ramkrishnapur Road, Barasat, North 24 Parganas, Kolkata – 700 125

Problem Statement

Write a Java program to create a simple form using Swing components for user input.

```
import javax.swing.*;
import java.awt.*;
import java.awt.event.*;

class SwingFormDemo implements ActionListener {

    JFrame frame;
    JLabel nameLabel, rollLabel;
    JTextField nameField, rollField;
    JButton submitButton;

    SwingFormDemo() {
        // Create a JFrame
        frame = new JFrame("Student Registration Form");

        // Create labels
        nameLabel = new JLabel("Name:");
        nameLabel.setBounds(50, 50, 100, 30);

        rollLabel = new JLabel("Roll No:");
        rollLabel.setBounds(50, 100, 100, 30);

        // Create text fields
        nameField = new JTextField();
        nameField.setBounds(150, 50, 150, 30);

        rollField = new JTextField();
        rollField.setBounds(150, 100, 150, 30);

        // Create button
        submitButton = new JButton("Submit");
        submitButton.setBounds(150, 150, 100, 30);

        // Add action listener to button
        submitButton.addActionListener(this);
    }
}
```



BRAINWARE UNIVERSITY

School of Engineering

Department of Computer Science and Engineering – Cyber Security & Data Science

398, Ramkrishnapur Road, Barasat, North 24 Parganas, Kolkata – 700 125

```
// Add components to frame
frame.add(nameLabel);
frame.add(rollLabel);
frame.add(nameField);
frame.add(rollField);
frame.add(submitButton);

// Set frame properties
frame.setSize(400, 300);
frame.setLayout(null);
frame.setVisible(true);
frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
}

public void actionPerformed(ActionEvent e) {
    String name = nameField.getText();
    String roll = rollField.getText();
    JOptionPane.showMessageDialog(frame, "Name: " + name + "\nRoll No: " + roll);
}

public static void main(String[] args) {
    new SwingFormDemo();
}
}
```

31. Use Case Scenario

In a student management system, new student records need to be added to a database for storage and future retrieval. Using JDBC (Java Database Connectivity), the program connects to a database and inserts data into a table.

Problem Statement

Write a Java program to insert student records into a database using JDBC.



BRAINWARE UNIVERSITY

School of Engineering

Department of Computer Science and Engineering – Cyber Security & Data Science

398, Ramkrishnapur Road, Barasat, North 24 Parganas, Kolkata – 700 125

```
import java.sql.*;

class JDBCInsertDemo {

    public static void main(String[] args) {
        // Database credentials and URL
        String url = "jdbc:mysql://localhost:3306/college"; // Replace 'college' with your database
        String user = "root"; // Your DB username
        String password = "password"; // Your DB password

        try {
            // Load JDBC driver
            Class.forName("com.mysql.cj.jdbc.Driver");

            // Establish connection
            Connection con = DriverManager.getConnection(url, user, password);

            // Create SQL INSERT statement
            String sql = "INSERT INTO students (roll_no, name, age) VALUES (?, ?, ?)";

            // Create PreparedStatement
            PreparedStatement ps = con.prepareStatement(sql);
            ps.setInt(1, 101);
            ps.setString(2, "Amit");
            ps.setInt(3, 20);

            // Execute the statement
            int rows = ps.executeUpdate();
            if (rows > 0) {
                System.out.println("Record inserted successfully.");
            }

            // Close resources
            ps.close();
            con.close();

        } catch (ClassNotFoundException e) {
            System.out.println("JDBC Driver not found.");
        } catch (SQLException e) {
            System.out.println("Database error: " + e.getMessage());
        }
    }
}
```




BRAINWARE UNIVERSITY

School of Engineering

Department of Computer Science and Engineering – Cyber Security & Data Science

398, Ramkrishnapur Road, Barasat, North 24 Parganas, Kolkata – 700 125

```
}  
}  
}
```

32. Use Case Scenario

In a student management system, the administrator needs to view all student records stored in the database. Using JDBC, the program connects to the database and retrieves data from the table for display.

Problem Statement

Write a Java program to read student records from a database using JDBC and display them on the console.

```
import java.sql.*;  
  
class JDBCReadDemo {  
  
    public static void main(String[] args) {  
        // Database credentials and URL  
        String url = "jdbc:mysql://localhost:3306/college"; // Replace 'college' with your DB name  
        String user = "root"; // DB username  
        String password = "password"; // DB password  
  
        try {  
            // Load JDBC driver  
            Class.forName("com.mysql.cj.jdbc.Driver");  
  
            // Establish connection  
            Connection con = DriverManager.getConnection(url, user, password);  
  
            // Create SQL SELECT statement  
            String sql = "SELECT roll_no, name, age FROM students";  
  
            // Create Statement  
            Statement stmt = con.createStatement();  
  
            // Execute query
```



BRAINWARE UNIVERSITY

School of Engineering

Department of Computer Science and Engineering – Cyber Security & Data Science

398, Ramkrishnapur Road, Barasat, North 24 Parganas, Kolkata – 700 125

```
ResultSet rs = stmt.executeQuery(sql);

// Display results
System.out.println("Roll No\tName\tAge");
System.out.println("-----");
while (rs.next()) {
    int roll = rs.getInt("roll_no");
    String name = rs.getString("name");
    int age = rs.getInt("age");
    System.out.println(roll + "\t" + name + "\t" + age);
}

// Close resources
rs.close();
stmt.close();
con.close();

} catch (ClassNotFoundException e) {
    System.out.println("JDBC Driver not found.");
} catch (SQLException e) {
    System.out.println("Database error: " + e.getMessage());
}
}
```

33. Use Case Scenario

In a web-based application, a program needs to connect to a web resource and retrieve information such as the protocol, host, and content. The URL class in Java provides a simple way to access web resources over the internet.

Problem Statement

Write a Java program to demonstrate networking using the URL class by connecting to a web resource and displaying its details.

```
import java.net.*;
import java.io.*;

class URLEDemo {
```



BRAINWARE UNIVERSITY

School of Engineering

Department of Computer Science and Engineering – Cyber Security & Data Science

398, Ramkrishnapur Road, Barasat, North 24 Parganas, Kolkata – 700 125

```
public static void main(String[] args) {
    try {
        // Create a URL object
        URL url = new URL("https://www.example.com");

        // Display URL information
        System.out.println("Protocol: " + url.getProtocol());
        System.out.println("Host: " + url.getHost());
        System.out.println("Port: " + url.getPort());
        System.out.println("File: " + url.getFile());

        // Open a stream to read data from URL
        BufferedReader br = new BufferedReader(new InputStreamReader(url.openStream()));
        String line;
        System.out.println("\nContent of the URL:");
        while ((line = br.readLine()) != null) {
            System.out.println(line);
        }
        br.close();

    } catch (MalformedURLException e) {
        System.out.println("Invalid URL: " + e.getMessage());
    } catch (IOException e) {
        System.out.println("I/O Error: " + e.getMessage());
    }
}
```

34. Use Case Scenario

In a networked application, it is often necessary to obtain information about a host, such as its IP address or hostname. The InetAddress class in Java provides methods to retrieve such information, which is useful for network diagnostics and connectivity checks.

Problem Statement

Write a Java program to demonstrate networking using the InetAddress class by finding the IP address and hostname of a machine or a website.



BRAINWARE UNIVERSITY

School of Engineering

Department of Computer Science and Engineering – Cyber Security & Data Science

398, Ramkrishnapur Road, Barasat, North 24 Parganas, Kolkata – 700 125

```
import java.net.*;

class InetAddressDemo {

    public static void main(String[] args) {
        try {
            // Get InetAddress object for a website
            InetAddress address = InetAddress.getByName("www.google.com");

            // Display host information
            System.out.println("Host Name: " + address.getHostName());
            System.out.println("IP Address: " + address.getHostAddress());

            // Get local host information
            InetAddress local = InetAddress.getLocalHost();
            System.out.println("\nLocal Host Name: " + local.getHostName());
            System.out.println("Local IP Address: " + local.getHostAddress());

        } catch (UnknownHostException e) {
            System.out.println("Host not found: " + e.getMessage());
        }
    }
}
```



BRAINWARE UNIVERSITY

School of Engineering

Department of Computer Science and Engineering – Cyber Security & Data Science

398, Ramkrishnapur Road, Barasat, North 24 Parganas, Kolkata – 700 125

HARDWARE AND SOFTWARE REQUIREMENTS

HARDWARE REQUIREMENTS (Minimum):

- Intel Pentium IV 1 GHz Processor
- Intel chipset 945 Motherboard
- 14” color Monitor
- Mouse
- Keyboard
- 2GB HDD
- 1GB RAM

SOFTWARE REQUIREMENTS:

- JDK & JRE
- NetBeans IDE



BRAINWARE UNIVERSITY

School of Engineering

Department of Computer Science and Engineering – Cyber Security & Data Science

398, Ramkrishnapur Road, Barasat, North 24 Parganas, Kolkata – 700 125

ASSIGNMENT SUBMISSION FORMAT

- 1. Front Page**
- 2. Content**
- 3. Assignment Number**
- 4. Problem Statement**
- 5. Program Code**
- 6. Input**
- 7. Output**



BRAINWARE UNIVERSITY

School of Engineering

Department of Computer Science and Engineering – Cyber Security & Data Science

398, Ramkrishnapur Road, Barasat, North 24 Parganas, Kolkata – 700 125

SAMPLE FRONT PAGE



BRAINWARE UNIVERSITY

School of Engineering

Department of Computer Science and Engineering – Cyber Security & Data Science

398, Ramkrishnapur Road, Barasat, North 24 Parganas, Kolkata – 700 125

LABORATORY ASSIGNMENT TOP SHEET

Subject:

Paper Code:

Assignment Title:

Assignment Date:

Date Submitted on:

Name:

Section:

Roll No:

Teacher's Signature:

Date:

Remarks:

